Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 04 - VHDL - Entwurf digitaler Schaltungen

Prof. Dr.-Ing. Guillermo Payá Vayá, 10. Dezember 2024

Arithmetik mit VHDL

Grundlage: Codes von Zahlen

Zahl: Zuordnung eines Wertes zu einem Codewort

Stellenwertverfahren

Zahl	$A = a_{n-1}a_{n-2}\ldots a_1 a_0$
Ziffer	$a_i \in \{0,1,\ldots,r-1\}$
Stellenwert	$w(i)$
Wert	$V(A) = \sum_{i=0}^{n-1} a_i w(i)$

Grundlagen: Polyadische Zahlensysteme

$$w(i) = r^i$$
$$V(A) = \sum_{i=0}^{n-1} a_i r^i$$

r Basis (Radix)

Name	Basis	Ziffernvorrat	Kennbuchstabe
dual (natürliches Binärsystem)	2	{0,1}	B
oktal	8	{0,1, ... 7}	O
dezimal	10	{0,1, ... 9}	D
hexadezimal	16	{0,1, ... 9, A, ... F}	H

Kennzeichnung der Basis $A_r = (a_{n-1}a_{n-2}a_1a_0)_r$

Grundlagen: Zweier-Komplement

n-stellige positive Zahl $A = 0\, a_{n-2} \ldots a_1\, a_0$

Zweier-Komplement $\overline{A}_2 = 1\, \overline{a}_{n-2} \ldots \overline{a}_1\, \overline{a}_0 + 1$

Summe:

$$
\begin{array}{rl}
A + \overline{A}_2 = & 0\, a_{n-2} \ldots a_1\, a_0 \\
& 1\, \overline{a}_{n-2} \ldots \overline{a}_1\, \overline{a}_0 \\
& \qquad\qquad\qquad 1 \\
\hline
& 1\ 1\ \ldots\ 1\ 1 \\
& \qquad\qquad\qquad 1 \\
\hline
& 1\ 0\ 0\ \ldots\ 0\ 0 \quad = 2^n
\end{array}
$$

Zweier-Komplement $\overline{A}_2 = 2^n - A$

Arithmetik-Datentypen (I)

- Arithmetische Operationen lassen sich in VHDL mit den folgenden Datentypen ausführen:
  - Standardtypen:
    - `integer` Ganze Zahlen: -2^{31} bis $2^{31} - 1$
    - `real` Fließkommazahlen: -10^{38} bis 10^{38}
      Sind meist nicht synthesefähig!!
  - `numeric_std`:

```
type signed is array (natural range <>) of std_logic;
type unsigned is array (natural range <>) of std_logic;
```

- Im Vergleich zum Datentyp `integer` sind die Datentypen `signed` bzw. `unsigned` inbesondere für Syntheseanwendungen sehr vorteilhaft, da sie auf dem Datentyp `std_(u)logic_vector` basieren und somit die einzelnen Bits einer Zahl als Digitalwert angesprochen werden können.

Arithmetik-Datentypen (II)

- Während viele Anwendungen mit Ganzzahldaten arbeiten, gibt es eine wachsende Anzahl von Anwendungen, die auch mit Festkomma- und Gleitkommazahlen umgehen.
  - Festkommazahlen (engl. Fixed-Point) (`fixed_pkg`) :

```
type sfixed is array (integer range < > ) of std_logic;
type ufixed is array (integer range < > ) of std_logic;

signal a : sfixed(7 downto -8);
signal b : ufixed(7 downto -8);
```

  - Gleitkommazahlen (engl. Floating-Point) (`float_pkg`):

```
type float is record
  sign      : std_logic;                        -- The sign bit (0 for positive, 1 for negative)
  exponent  : unsigned(EXP_WIDTH-1 downto 0);   -- Exponent field
  fraction  : unsigned(FRAC_WIDTH-1 downto 0);  -- Fraction (or mantissa) field
end record;

signal a : float; -- A floating-point signal using the default "float" type
```


Datentyp: `signed` und `unsigned`

- Das `numeric_std`-Package definiert beide Typen (`signed` und `unsigned`), die unbeschränkte Arrays des Elementartypen `std_logic` sind.

```
type signed is array (natural range <>) of std_logic;
type unsigned is array (natural range <>) of std_logic;
```

- Zu beachten ist, dass der Index vom Typ `natural` ist, wodurch keine negativen Indizes erlaubt sind.
- Beispiel:

```
signal a : signed(7 downto 0);
signal b : unsigned(7 downto 0);
```

Vergleichsoperatoren

Vergleichsoperator	Bedeutung	Beispiel
=	gleich	`... when (A = B) ...`
/=	ungleich	`... when (A /= B) ...`
<	kleiner	`... when (A < B) ...`
<=	kleiner oder gleich	`... when (A <= B) ...`
>	größer	`... when (A > B) ...`
>=	größer oder gleich	`... when (A >= B) ...`

- In Synthesewerkzeugen werden Operatoren als Schaltungsmakros identifiziert und durch diese ersetzt, sofern für das jeweilige Makro zur Verfügung gestellte Entsprechung in der Entwurfsbibliothek verfügbar ist.
- Der Operator wird in logische Grundelemente aufgelöst, falls kein Bibliothekselement existiert.

Vergleichs-operatoren

Arithmetik-Operatoren

Operator	Bedeutung	Beispiel	Synthesefähigkeit
+	Addition	`Y <= A + B`	ja
-	Subtraktion	`Y <= A - B`	ja
abs	Absolutwertbildung	`Y <= abs(A)`	ja
*	Multiplikation	`Y <= A * B`	ja
/	Division	`Y <= A / B`	ja/nein[1]
**	Potenzbildung	`Y <= 2**B`	ja/nein[2]
mod	Rest der Division[3]	`Y <= A mod B`	ja/nein[1]
rem	Rest der Division[4]	`Y <= A rem B`	ja/nein[1]

[1] synthesefähig falls B eine Zweierpotenz
[2] nur Potenzen von 2 erlaubt (Linksverschiedung einer Binärzahl)
[3] **mod** → Rest der Division A/B. A mod B = A - B*n;
(n ist der ganzzahlige Teil der Division). Vorzeichen des Ergebnisses ist das von B
[4] **rem** → Rest der Division A/B. A rem B = A - (A/B)*B;
Vorzeichen des Ergebnisses ist das von A

Arithmetik-operatoren

Schiebe-Operatoren

Operator	Bedeutung	Beispiel	Syn.
`shift_left()`	Links schieben um N Bit	`Y<=shift_left(A,3)`	ja[1]
`shift_right()`	Rechts schieben um N Bit	`Y<=shift_right(A,3)`	ja[2]
`rotate_left()`	Links rotieren um N Bit	`Y<=rotate_left(A,3)`	ja[3]
`rotate_right()`	Rechts rotieren um N Bit	`Y<=rotate_left(A,3)`	ja[4]

[1] die höchstwertigen Bits gehen verloren, rechts wird mit Nullen aufgefüllt
[2] die niederwertigen Bits gehen verloren, links wird vorzeichengerecht ergänzt
[3] die links heraus geschobenen Bits werden rechts hinein geschoben
[4] die rechts heraus geschobenen Bits werden re links hinein geschoben

Schiebe-operatoren

Konvertierungsfunktionen aus `ieee.numeric_std`

- Notwendigkeit:
  - VHDL ist eine stark typisierte Sprache!!
- Vorhandene Funktionen:

Funktionen

Konvertierungsfunktion	Arg.	Size	Ergebnistyp
`to_integer(ARG)`	`signed` / `unsigned`	-	`integer`
`unsigned(ARG)`	`signed`	-	`unsigned`
`unsigned(ARG)`	`std_(u)logic_vector`	-	`unsigned`
`to_unsigned(ARG,SIZE)`	`natural`	Anzahl der Bits	`unsigned`
`signed(ARG)`	`unsigned`	-	`signed`
`signed(ARG)`	`std_(u)logic_vector`	-	`signed`
`to_signed(ARG,SIZE)`	`integer`	Anzahl der Bits	`signed`
`resize(ARG,SIZE)`	`signed`	Anzahl der Bits	`signed`
`resize(ARG,SIZE)`	`unsigned`	Anzahl der Bits	`unsigned`
`std_logic_vector(ARG)`	`signed` / `unsigned`	-	`std_logic_vector`
`std_ulogic_vector(ARG)`	`signed` / `unsigned`	-	`std_ulogic_vector`

Beispiel: Komparator

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity KOMPARATOR is
port( A, B:          in  unsigned(3 downto 0);
      KL, GL, GR:    out std_ulogic);
end KOMPARATOR;
architecture RTL of KOMPARATOR is
signal TEMP: std_ulogic_vector(2 downto 0);
begin
  P1: process(A, B)
  begin
    TEMP <= "000";
    if (A > B) then    TEMP <= "100";
    elsif (A < B) then TEMP <= "010";
    else               TEMP <= "001";
    end if;
  end process P1;
    GL <= TEMP(0);
    KL <= TEMP(1);
    GR <= TEMP(2);
end architecture RTL;
```

Komparator

Beispiel: Synchronzähler

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity ZAEHLER is
  generic( BITS: natural := 4);
  port( clk, rst_n, load : in std_ulogic;
        D : in  unsigned(BITS-1 downto 0);
        Q : out unsigned(BITS-1 downto 0);
        overflow : out std_ulogic);
end ZAEHLER;
architecture RTL of ZAEHLER is
  signal Q_ff : unsigned(BITS-1 downto 0);
begin
  counter: process (clk, rst_n)
  begin
    if rst_n ='0' then
      Q_ff <= (others =>'0');
    elsif clk='1' and clk'event then
      if load = '1' then Q_ff <= D;
      else Q_ff <= Q_ff + 1;
      end if;
    end if;
  end process counter;
  overflow <= '1' when (Q_ff = 2**BITS-1) else '0';
  Q <= Q_ff;
end architecture RTL;
```

Synchron-zähler

Beispiel: Konvertierungsfunktionen

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity ADD_SUB is
port( A_i ,B_i    : in  std_ulogic_vector(4 downto 0);
      OP_i, C_i   : in  std_ulogic;
      SUM_o       : out std_ulogic_vector(4 downto 0));
end ADD_SUB;
architecture RTL of ADD_SUB is
begin
  ADDSUB : process(OP_i, C_i, A_i, B_i)
    variable ZW, V1, V2, V3: signed(4 downto 0);
  begin
    V1 := signed(A_i(4 downto 0));
    V2 := signed(B_i(4 downto 0));
    V3 := "0000" & C_i;
    if OP_i = '1' then
      ZW := V1 + V2 + V3; -- ADDITION
    else
      ZW := V1 - V2 - V3; -- SUBTRAKTION
    end if;
    SUM_o <= std_ulogic_vector(ZW);
  end process ADD_SUB;
end architecture RTL;
```

Konvertieren

Integer-Arithmetik

- `integer` waren ursprünglich nur für Simulationszwecke vorgesehen.
- Die Implementierungen sind meist sehr Synthesewerk spezifisch.
- *Heutzutage ist die Verwendung des integer-Datentyps für Syntheseanwendungen nur noch erforderlich, wenn indiziert auf Vektor- oder Feldkomponenten zugegriffen und wenn Parameter in Strukturmodellen an Komponenten übergeben werden sollen.*
- Weitere Anwendungen dieses Datentyps sollten auf reine Verhaltensmodelle bzw. auf Testumgebungen beschränkt bleiben.
- Zu dem `integer` Datentyp existieren zwei vordefinierte weitere Datentypen, `natural` und `positive`, die als subtype (abgeleitete Datentypen) bezeichnet werden:

```
signal BYTE : integer range 0 to 255;

subtype natural is integer range 0 to 2147483647;
subtype positive is integer range 1 to 2147483647;
```

Beispiel: Speicher / Memory

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity RAM is
  generic (word_size_g    : natural;
           address_size_g : natural);
  port (clk, write_i      : in  std_ulogic;
        address_i         : in  std_ulogic_vector(address_size_g-1 downto 0);
        D_i               : in  std_ulogic_vector(word_size_g-1 downto 0);
        Q_o               : out std_ulogic_vector(word_size_g-1 downto 0));
end entity RAM;
architecture RTL of RAM is
  type memory_type_t is array (0 to 2**address_size_g-1) of
    std_ulogic_vector(word_size_g-1 downto 0);
  signal memory : memory_type_t;
begin
  speicher_inst : process (clk)
  begin
    if clk = '1' and clk'event then
      if write_i = '1' then
        memory(to_integer(unsigned(address_i))) <= D_i;
      end if;
    end if;
  end process speicher_inst;
  Q_o <= memory(to_integer(unsigned(address_i)));
end architecture RTL;
```

Speicher

Beispiel: `for`-Anweisungen

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity PATTERN_FINDER is
  generic (pattern_size_g, input_size_g   : natural);
  port ( pattern_i          : in  std_ulogic_vector(pattern_size_g-1 downto 0);
         data_i             : in  std_ulogic_vector(input_size_g-1 downto 0);
         found_o            : out std_ulogic);
end entity PATTERN_FINDER;
architecture RTL of PATTERN_FINDER i
begin
  pattern_search : process (pattern_i, data_i)
    variable match: std_ulogic_vector(pattern_size_g -1 downto 0) := (others => '0');
    variable found: std_ulogic := '0';
  begin
    found := '0';
    for i in 0 to input_size_g - pattern_size_g loop
      match := (others => '0');
      for j in 0 to pattern_size_g-1 loop
        match(j) := '1' when data_i(i+j) = pattern_i(j) else '0';
      end loop;
      if and(match) then found := '1'; end if;
    end loop;
    found_o <= found;
  end process pattern_search;
end architecture RTL;
```

For-Loop

Grundlagen: Gebrochene Zahlen

- Zahl $A = a_{n-1}a_{n-2}\ldots a_1 a_0, a_{-1}a_{-2}\ldots a_{-m}$
- Wert $V(A) = \sum_{i=-m}^{n-1} a_i r^i$

Fixed-Point-Arithmetik (I)

- Das Package `fixed_pkg` bietet Festkommaarithmetik mit beliebiger Genauigkeit.
- Das Package wurde in VHDL-2008 eingeführt.
- Die Festkomma-Arithmetik ist eine Zwischenform der Arithmetik zwischen der Ganzzahl- und der Gleitkomma-Arithmetik.
- Es erzeugt in der Regel gleich große/schnelle oder etwas größere/langsamere Schaltungen als Ganzzahl-Arithmetik.
- Festkomma-Arithmetik wird häufig in Anwendungen wie der digitalen Signalverarbeitung (DSP) verwendet. Eine wichtige Herausforderung bei der Hardwareentwicklung besteht darin, einen Algorithmus, der in Gleitkomma ausgedrückt ist, in eine Festkomma-Implementierung umzuwandeln, um die Chipgröße zu verringern und die Leistung zu erhöhen und dabei die durch Fixed-Point verursachten Berechnungsfehler zu minimieren.

Fixed-Point-Arithmetik (II)

- Mit dem `fixed-point` Package kann die Position des Binärpunkts im Entwurf angegeben und in der Simulationsausgabe visualisiert werden.
- Die Festkomma-Arithmetik ist ähnlich implementiert wie die Ganzzahl-Arithmetik, mit zusätzlicher Schaltung zur Unterstützung von Rundungs- und Überlaufmodi. Es ist möglich, diese Modi so zu wählen, dass kein Overhead entsteht.
- Das `fixed-point` Package wird weitgehend mit `numeric_std` implementiert und profitiert daher von den Optimierungen, die Synthesewerkzeuge bereits für dieses Paket bieten.

```
type sfixed is array (integer range < > ) of std_logic;
type ufixed is array (integer range < > ) of std_logic;

signal a : sfixed(7 downto -8);
signal b : ufixed(7 downto -8);
```


Konvertierungsfunktionen

Konvertierungsfunktion	Arg.	Ergebnistyp
`to_sfixed(ARG, high, low)`[1]	`real` / `integer`	`sfixed`
`to_ufixed(ARG, high, low)`[1]	`real` / `integer`	`ufixed`
`resize(ARG, high, low)`[1]	`sfixed` / `ufixed`	wie Eingabe
`to_sfixed(ARG, high, low)`[1]	`std_(u)logic_vector`	`sfixed`
`to_ufixed(ARG, high, low)`[1]	`std_(u)logic_vector`	`ufixed`
`to_sfixed(ARG, high, low)`[1]	`signed`	`sfixed`
`to_ufixed(ARG, high, low)`[1]	`unsigned`	`ufixed`
`to_unsigned(ARG, size)`[2]	`ufixed`	`unsigned`
`to_signed(ARG, size)`[2]	`sfixed`	`signed`
`to_real(ARG)`	`sfixed` / `ufixed`	`real`
`to_integer(ARG)`[2]	`sfixed` / `ufixed`	`real`
`to_slv(ARG)`[3]	`sfixed` / `ufixed`	`standard_logic_vector`
`to_sulv(ARG)`[3]	`sfixed` / `ufixed`	`standard_logic_vector`

[1] Statt `high` und `low` auch Signal mit entsprechende Größe
[2] Es kann ein `round_style` und ein `saturate_style` angegeben werden.
[3] Die Anzahl der Bits im Ergebnis ist gleich derjenigen im Ursprungswert, die Nummerierung wird jedoch verschoben.

Konvertieren

Beispiel: Addierer (I)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.fixed_pkg.all;
entity ADDER is
  port(A_i   : in  sfixed(3 downto -7);
       B_i   : in  sfixed(5 downto -4);
       SUM_o : out sfixed(5 downto -4));
end ADDER;
architecture RTL of ADDER is
begin

  SUM_o <= A_i + B_i; -- ERROR !!

  SUM_o <= resize(A_i(3 downto -4) + B_i, SUM_o);

  SUM_o <= resize(A_i(3 downto -4) + B_i, SUM_o'left, SUM_o'right); -- alternativ

end architecture RTL;
```

Addierer

Beispiel: Addierer (II)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.fixed_pkg.all;
use IEEE.fixed_float_types.all;

entity ADDER is
  port(A_i   : in  sfixed(3 downto -7);
       B_i   : in  sfixed(5 downto -4);
       SUM_o : out sfixed(5 downto -4));
end ADDER;
architecture RTL of ADDER is
begin

  SUM_o <= resize(A_i(3 downto -4) + B_i, SUM_o, fixed_wrap, fixed_truncate);

  SUM_o <= resize(A_i(3 downto -4) + B_i, SUM_o'left, SUM_o'right, fixed_saturate, fixed_round);

end architecture RTL;
```

Addierer

- Überlaufmodi (Overflow)
  - Wrap mode (`fixed_wrap`)
  - Saturation mode (`fixed_saturate`)
- Rundung (Underflow)
  - Rounding mode (`fixed_round`)
  - Truncate mode (`fixed_truncate`)

Beispiel: Multiplikation

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.fixed_pkg.all;
entity MULTIPLIER is
  port(A_i   : in  sfixed(3 downto -7);
       B_i   : in  sfixed(5 downto -4);
       MUL_o : out sfixed(8 downto -4));
end MULTIPLIER;
architecture RTL of MULTIPLIER is
begin

  MUL_o <= A_i * B_i; -- ERROR !!

  MUL_o <= resize(A_i(3 downto -4) * B_i, MUL_o);

  MUL_o <= resize(A_i(3 downto -4) * B_i, MUL_o'left, MUL_o'right); -- alternativ

end architecture RTL;
```

Multiplizierer

Beispiel: Konvertierungsfunktionen

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.fixed_pkg.all;
use IEEE.fixed_float_types.all;

entity mm_to_inches is
    port(
        mms : in unsigned(15 downto 0); -- mm Input
        inches : out std_ulogic_vector(7 downto 0) --  Inch Output
    );
end mm_to_inches;

architecture behavior of mm_to_inches is
begin
    process(mms)
      variable ufixed_mm : ufixed(15 downto -12);
      variable result : ufixed(24 downto -16);
    begin
      -- Konvertierung von unsigned zu ufixed
      ufixed_mm := to_ufixed(mms, ufixed_mm);
      -- ufixed kann mit real-Konstanten multipliziert werden
      result := resize(ufixed_mm / 25.4, result);
      -- resize auf Ausgabegröße
      inches <= to_std_ulogic_vector(resize(result, 7, 0, fixed_wrap, fixed_truncate));
    end process;
end behavior;
```

Konvertieren

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.